



Facultad Ingeniería Informática
Universidad Tecnológica de La Habana
José Antonio Echeverría
cujae

Informe del Proyecto Final de la Asignatura **Diseño y Programación Orientada a** **Objetos**



Integrantes del equipo SchenellerXIII:

Lian Carlos Gené Galvez G13

Marlon Miguel González Llerandi G13

1. Enunciado de la problemática

El Vicedecanato de Investigaciones y Posgrado de la facultad desea llevar un control informático de la actividad investigativa y de posgrado de los profesores de la facultad. Para ello, se lleva el control de los docentes de la facultad que participan en las diferentes líneas de investigación y de los que imparten y reciben los cursos de posgrado que se ofertan en la facultad.

De cada docente se conoce que tiene una categoría docente (Instructor, Asistente, Profesor Auxiliar o Profesor Titular) y puede tener o no una categoría científica (Máster o Doctor).

La actividad investigativa de la facultad se organiza en diferentes líneas de investigación, donde cada una tiene un responsable a su cargo, que es docente, y está integrada por docentes y estudiantes, cada uno de los cuales trabaja en un tema de investigación determinado

Los resultados de cada investigador se miden por los artículos que publican en diferentes revistas, por los capítulos de libros, o por las ponencias en diferentes eventos científicos. En ese sentido, cada artículo tiene un título, número, volumen, año de publicación, páginas y grupo de impacto (WoS, Grupo 1, Grupo 2, Grupo 3 y Grupo 4). Los capítulos de libros tienen un título, autor(es), nombre de los editores, editorial y código ISSN, nombre del libro y volumen. Por su parte, de cada ponencia en evento se tiene un nombre, un año, lugar, fecha y un ISBN para la publicación de las ponencias. Para calcular el puntaje de un investigador, según sus resultados investigativos, se tiene que los artículos en revista aportan 10 puntos si es WoS, 8 si es Grupo 1, 6 si es Grupo 2 y 4 si es Grupo 3 o 4, los capítulos de libro aportan 3 puntos y los eventos aportan 1 punto.

La facultad también oferta cursos de posgrado para los docentes. Estos cursos son impartidos por un docente con categoría científica "Doctor", tienen un tema, objetivos y otorgan una cierta cantidad de créditos a los que lo reciben. El profesor que imparte el curso emite una nota para cada uno de los participantes en dicho curso. El Plan de Estudios de la Maestría agrupa a varios de estos cursos de posgrado. Algunos docentes de la facultad cursan la maestría, para lo cual deben estar matriculados. Los docentes matriculados reciben el visto bueno del vicedecanato para defender dicha maestría, una vez que han obtenido todos los créditos de las asignaturas definidas en este Plan de Estudios.

De cada investigador se registran todos los artículos que ha publicado, así como los cursos de posgrado que ha impartido y los créditos obtenidos por cursos recibidos. El control incluye la actividad investigativa y de posgrado por departamento en el año.

2. Diagrama de clases

1. Vicedecanato ← Docente:

Relación: Composición.

Propósito: El Vicedecanato posee completamente a los profesores en cuanto a sus actividades de investigación y posgrado, si el vicedecanato deja de existir entonces el control sobre los docentes también desaparece, porque los docentes no tendrían sentido de existir en el sistema sin el control que maneja el Vicedecanato.

Cardinalidad: Un Vicedecanato está conformado por 10 o más docentes, con menos no garantiza un manejo consistente de la información deseada.

2. Vicedecanato ← Estudiante:

Relación: Composición.

Propósito: El Vicedecanato posee un control total de los estudiantes que van a participar en las líneas de investigación, sin eso, no tiene sentido registrar a los estudiantes.

Cardinalidad: Un vicedecanato debe estar conformado por 10 o más estudiantes que vayan a participar en las líneas de investigación, un caso similar al docente.

3. Vicedecanato ← Departamento:

Relación: Composición.

Propósito: Cada departamento debe ser gestionado por el Vicedecanato para organizar de manera jerárquica, la investigación y posgrado, por ello cada departamento depende del Vicedecanato que lo gestiona.

Cardinalidad: Un Vicedecanato debe tener al menos un departamento que organice toda la actividad investigativa y el posgrado.

4. Departamento ← Docente:

Relación: Agregación.

Propósito: El departamento organiza a los docentes, pero estos pueden existir independientemente. Y aunque desapareciera, el docente puede simplemente reasignarse a otro, hecho para organizar mejor la jerarquía de la información, además un docente solo puede pertenecer a un solo departamento, para estar en otro tiene que salir en el que actualmente permanece.

Cardinalidad: A un departamento se le pueden agregar de 0 a muchos docentes.

5. Departamento ← Estudiante:

Relación: Agregación.

Propósito: Similar a la relación con docentes, los estudiantes pueden ser reasignados si el departamento deja de existir, además un estudiante solo puede pertenecer a un solo departamento, para estar en otro tiene que salir en el que actualmente permanece.

Cardinalidad: A un departamento se le pueden agregar de 0 a muchos docentes.

6. Departamento ← LíneaInvestigación:

Relación: Composición.

Propósito: Las líneas de investigación están gestionadas y controladas por el departamento, si el departamento desaparece no tendría sentido que existieran sus líneas de investigación.

Cardinalidad: Un departamento debe estar conformado por 1 o más líneas de investigación.

7. Departamento ← Maestría:

Relación: Composición.

Propósito: Cada Maestría depende de un Departamento para su organización. Sin el departamento, la Maestría no podría existir en el sistema.

Cardinalidad: Un departamento debe tener 1 o más maestrías a las que gestionar.

8. LíneaInvestigación ← Investigador:

Relación: Agregación.

Propósito: Las líneas de investigación deben tener investigadores que lleven a cabo resultados de investigaciones, si un investigador abandona una línea simplemente puede unirse a otra, un investigador solo puede pertenecer a una línea de investigación a la vez.

Cardinalidad: Una línea de investigación tiene 0 o más investigadores.

9. ResultadoInvestigativo ← CapítuloLibro, PonenciaEvento, Artículo:

Relación: Herencia.

Propósito: Los resultados investigativos comparten el aporte del puntaje hacia el Investigador que los compone, donde cada uno le da la implementación de su preferencia, por lo que CapítuloLibro, PonenciaEvento y Artículo heredan de la clase abstracta ResultadoInvestigativo .

10. Investigador ← ResultadoInvestigativo:

Relación: Composición.

Propósito: El resultado investigativo forma parte esencial de la actividad del investigador, si el investigador se elimina sus resultados también.

Cardinalidad: Un investigador tiene 0 o más resultados investigativos.

11. Investigador $\leftarrow$ Estudiante, Docente:

Relación: Herencia.

Propósito: Tanto Estudiantes como Docentes son Investigadores, por lo que heredan sus atributos y métodos.

12. LíneaInvestigación $\leftarrow \rightarrow$ Docente:

Relación: Asociación.

Propósito: Por cada línea de investigación hay un encargado que debe ser un docente.

Cardinalidad: Una línea de investigación tiene un encargado (Docente).

13. Maestría $\leftarrow$ CursoPosgrado

Relación: Composición.

Propósito: La Maestría se compone de cursos de posgrado, por tanto, sin la Maestría, los cursos no tendrían sentido en el sistema.

Cardinalidad: Una maestría debe tener cursos, mínimo uno.

14. Maestría $\leftarrow$ Docente:

Relación: Agregación.

Propósito: Los docentes se pueden matricular en la maestría para cursarla, pero su existencia no depende de ella, solo pueden matricularse en una maestría a la vez.

Cardinalidad: La maestría puede tener varios docentes matriculados o ninguno.

15. Docente $\leftarrow \rightarrow$ CursoPosgrado:

Relación: Asociación.

Propósito: Un docente doctor imparte cursos de posgrado y emite una nota a los estudiantes del mismo.

Cardinalidad: El docente doctor puede impartir uno o varios cursos de posgrado.

16. Docente $\rightarrow$ CursoPosgrado:

Relación: Agregación.

Propósito: Los docentes participan en los cursos necesarios para poder defender su maestría.

Cardinalidad: Un docente puede participar en varios cursos o ninguno, después de todo, necesitan los créditos de los cursos que reciben para obtener el visto bueno del Vicedecanato y poder defender la Maestría.

17. CursoRecibido:

Esta clase se genera a partir de la asociación del Docente Doctor que imparte el curso con el curso impartido al emitir una nota a uno de los docentes participantes, agrupa la información de la nota recibida y los créditos recibidos si el participante con la nota emitida aprobó el curso y esta información se guarda dentro del docente para más tarde ser usada para verificar si el docente propietario de la instancia de esta clase tiene todos los créditos requeridos para defender la Maestría a la que está matriculado.

3. Argumento sobre la ausencia de interfaces

En el contexto de este modelado, no se requiere el uso de interfaces porque las relaciones y funcionalidades están bien definidas mediante la herencia y la composición. Las siguientes razones justifican esto:

La relación de herencia de Resultado Investigativo con Artículo, PonenciaEvento y CapituloLibro ya cubre el caso donde se necesita polimorfismo, lo que facilita el cálculo de puntajes sin una interfaz común. Además, según la problemática, el Estudiante y el Docente son Investigadores, por tanto, se les trata como tal en las líneas de investigación.

Las operaciones del sistema como calcular los puntajes y gestionar los cursos están encapsuladas en clases concretas con implementaciones directas. No hay comportamientos que deban ser abstraídos en interfaces ya que la herencia y las asociaciones explicadas anteriormente cubren eso.

Nuestro modelo prioriza la simplicidad, introducir interfaces desde nuestro punto de vista añadiría complejidad innecesaria ya que las clases tienen responsabilidades bien claras y no hay varias implementaciones de un comportamiento.

4. Responsabilidades

Lian Carlos → Implementación de clases y CRUD del control del posgrado y login: Encargado de implementar las clases y el CRUD de Vicedecanato, Departamento, Docente y Estudiante y las relacionadas con el control de la actividad de Posgrado(Maestría, Cursos...), responsable de la implementación del sistema de autenticación o login del programa, diseñador de la interfaz.

Marlon Miguel → Encargado de implementar las clases y el CRUD del control de la actividad investigativa (Líneas de Investigación, Investigador y los Resultados Investigativos).

5. Reportes

Lian Carlos:

1. Plan de Estudios de la Maestría: Describe los cursos de posgrado agrupados en el plan de estudios, con detalles de tema, objetivos, créditos y docentes que los imparten.
2. Estado de los docentes matriculados en la Maestría: Tabla de información sobre los docentes que están matriculados en una Maestría de uno de los departamentos, los créditos acumulados y si están listos para la defensa de la maestría.

Marlon Miguel:

3. Producción científica: Muestra las cantidades de resultados investigativos por tipo y el total en todas las líneas de investigación de un departamento seleccionado
4. Ranking de investigadores por puntaje acumulado: Clasifica a los investigadores según los puntos obtenidos por sus resultados investigativos a nivel de departamento.

6. Explicación de la implementación de las clases

- Vicedecanato

La clase Vicedecanato representa el núcleo administrativo del sistema. Es una clase Singleton, lo que garantiza que exista una sola instancia global encargada de gestionar docentes, estudiantes y departamentos. Su objetivo es centralizar la coordinación de entidades académicas bajo el control de un vicedecanato en una facultad universitaria.

Atributos

- ArrayList<Docente> docentes: Lista de todos los docentes registrados en el sistema.
- ArrayList<Estudiante> estudiantes: Lista de estudiantes registrados.
- ArrayList<Departamento> departamentos: Contenedor de los departamentos académicos de la facultad.
- static Vicedecanato vicedecanato: Instancia única (patrón Singleton).

Métodos y su propósito

Constructor privado

- private Vicedecanato(): Evita instanciación externa. Inicializa listas vacías.

Singleton

- getVicedecanato(): Devuelve la instancia única de la clase. Si no existe, la crea.

Métodos de inserción

- agregarDocente(Docente d): Agrega un docente, validando duplicados.
- agregarEstudiante(Estudiante e): Agrega un estudiante con verificación.
- agregarDepartamento(Departamento d): Agrega un departamento, sin duplicación.

Métodos de eliminación

- removerDocente(Docente d): Elimina un docente y lo desvincula de cualquier departamento.
- removerEstudiante(Estudiante e): Idéntico al anterior, para estudiantes.
- removerDepartamento(Departamento d): Elimina un departamento del registro.

Evaluación académica

- darVistoBuenoMaestria(Maestria m, Docente d): Verifica si un docente ha obtenido suficientes créditos para aprobar una maestría.

Reportes

- generarDatosReporte1(ArrayList<Investigador> ranking, Departamento depto): Genera un ranking de investigadores de un departamento en orden decreciente según su puntaje investigativo.

Creación de entidades

- crearDocente(...): Crea y agrega un nuevo docente.
- crearEstudiante(...): Crea y registra un estudiante.
- crearDepartamento(String nombre): Crea y agrega un departamento.

Verificación de pertenencia

- solicitudValidaIngresoDocenteADepartamento(Docente d): Verifica si un docente no está ya en un departamento.
- solicitudValidaIngresoEstudianteADepartamento(Estudiante e): Igual, para estudiantes.

Listados filtrados

- obtenerDocentesNoRegistradosEnDepartamentos(): Devuelve todos los docentes sin departamento.
- obtenerEstudiantesNoRegistradosEnDepartamentos(): Lo mismo, para estudiantes.

Localización

- obtenerDepartamentoDeUnDocente(Docente d): Devuelve el departamento al que pertenece un docente.
- obtenerDepartamentoDeUnEstudiante(Estudiante e): Devuelve el de un estudiante.

- Departamento

La clase Departamento representa una unidad académica dentro de una facultad universitaria. Es responsable de gestionar y organizar los docentes, estudiantes, maestrías y líneas de investigación que le pertenecen. Tiene como misión ser el contenedor directo de actividades académicas e investigativas especializadas.

Atributos

- String nombre: Nombre del departamento.
- ArrayList<Docente> docentes: Lista de docentes adscritos al departamento.
- ArrayList<Estudiante> estudiantes: Lista de estudiantes asignados al departamento.
- ArrayList<Maestria> maestrias: Maestrías que se imparten en el departamento.
- ArrayList<LineaInvestigacion> lineasInvestigacion: Conjunto de líneas de investigación asociadas.

Métodos y su propósito

Constructor

- Departamento(String nombre): Inicializa el departamento con un nombre válido y listas vacías. Valida que el nombre contenga solo letras y espacios.

Igualdad y representación

- equals(Object o): Compara departamentos por nombre.
- toString(): Devuelve el nombre del departamento como cadena.

Gestión de docentes

- agregarDocente(Docente d): Añade un docente si no está repetido.
- removerDocente(Docente d): Elimina al docente y lo desvincula de todas las maestrías y líneas de investigación donde esté inscrito.
- contieneDocente(Docente d): Verifica si un docente pertenece al departamento.

Gestión de estudiantes

- agregarEstudiante(Estudiante e): Añade un estudiante si no está repetido.
- removerEstudiante(Estudiante e): Elimina al estudiante y lo desvincula de cualquier línea de investigación.
- contieneEstudiante(Estudiante e): Comprueba si un estudiante forma parte del departamento.

Gestión de maestrías

- agregarMaestria(Maestria m): Añade una maestría si no está duplicada.
- removerMaestria(Maestria m): Elimina una maestría del departamento.
- crearMaestria(...): Crea y añade una nueva maestría.

Gestión de líneas de investigación

- agregarLineaInvestigacion(LineaInvestigacion l): Añade una línea nueva si no existe.

- `removerLineaInvestigacion(LineaInvestigacion l)`: Elimina una línea existente.
- `crearLineaInvestigacion(...)`: Crea una nueva línea con un responsable docente.

Verificaciones y listados

- `solicitudValidaIngresoInvestigadorALineaDeInvestigacion(Investigador inv)`: Verifica si un investigador no pertenece aún a ninguna línea.
- `obtenerInvestigadoresNoRegistradosLineasDeInvestigacion()`: Devuelve investigadores (docentes y estudiantes) que no están en ninguna línea y no sean responsables de ninguna.
- `solicitudValidaMatriculaMaestria(Docente d)`: Comprueba si un docente no está ya matriculado en una maestría.
- `obtenerListaDocentesValidosParaMatriculaMaestria()`: Devuelve docentes que no están en maestrías y no poseen categoría científica.

Búsqueda y relación

- `buscarInvestigadorDelResultado(ResultadoInvestigativo r)`: Retorna el investigador responsable de un resultado específico.
- `obtenerMaestriaDeUnCurso(CursoPosgrado c)`: Busca a qué maestría pertenece un curso de posgrado.
- `obtenerListaDocentesValidosParaSerEncargadosEnNuevaLineaDeInvestigacion()`: Devuelve la lista de docentes que son válidos para ser encargados de una línea de investigación.
- Docente

La clase `Docente` representa a un profesional universitario que tiene funciones académicas y científicas. Hereda de la clase `Investigador`, lo que le permite participar en líneas de investigación y producir resultados investigativos. Además, tiene la capacidad de impartir y recibir cursos de posgrado, y se caracteriza por poseer una categoría científica y docente.

Atributos

- `CategoriaCientifica catCientifica`: Categoría científica del docente (Ej. Doctor, Máster, Ninguna).
- `CategoriaDocente catDocente`: Nivel profesional dentro del cuerpo docente (Ej. Asistente, Titular, Auxiliar).
- `ArrayList<CursoPosgrado> cursosImpartidos`: Cursos de posgrado que el docente ha impartido.
- `ArrayList<CursoRecibido> cursosRecibidos`: Cursos que el docente ha recibido como parte de su formación continua.

Métodos y su propósito

Constructor

- `Docente(String nombre, String apellidos, CategoriaCientifica catCientifica, CategoriaDocente catDocente)`: Inicializa los datos básicos heredados y define las categorías profesional y científica. Valida que estas no sean nulas.

Gestión de cursos

- `agregarCursoImpartido(CursoPosgrado c)`: Permite añadir un curso impartido si el docente tiene categoría de Doctor. Valida que no esté duplicado.
- `removeCursoImpartido(CursoPosgrado c)`: Elimina un curso previamente impartido. Requiere que no sea null, que la lista no esté vacía y que el curso exista en ella.
- `agregarCursoRecibido(CursoRecibido c)`: Añade un curso a la lista de recibidos. Verifica que no sea nulo ni esté duplicado.
- `removeCursoRecibido(CursoRecibido c)`: Elimina un curso recibido de la lista de cursos siempre y cuando la lista no esté vacía y el curso que se desea eliminar esté presente en la lista.
- `contieneRegistroCursoRecibido(CursoRecibido cursoRecibido)`: Verifica que el curso del parámetro ya esté registrado.

Igualdad y representación

- `equals(Object d)`: Compara docentes por nombre, apellidos, categoría científica y docente.
- `toString()`: Devuelve una representación textual del docente (nombre completo).

- Estudiante

La clase Estudiante representa a un alumno universitario con participación activa en actividades investigativas. Hereda de la clase Investigador, lo que le permite integrarse a líneas de investigación y asociarse a resultados científicos. Su principal característica distintiva es el atributo grupo, que indica su pertenencia académica dentro de la institución.

Atributos

- `String grupo`: Identificador del grupo académico al que pertenece el estudiante (por ejemplo, grupo de clase o curso).

Métodos y su propósito

Constructor

- `Estudiante(String nombre, String apellidos, String grupo)`: Crea un nuevo estudiante con nombre, apellidos y grupo. Hereda nombre y apellidos desde la clase `Investigador`. Valida que el grupo no sea una cadena vacía.

Métodos adicionales

- `equals(Object o)`: Compara dos estudiantes por nombre, apellidos y grupo, asegurando una comparación lógica y coherente.
- `toString()`: Devuelve una representación textual del estudiante con su nombre completo.

- `Investigador`

La clase `Investigador` es una clase abstracta que define la estructura y el comportamiento común de todas las personas que participan en actividades investigativas dentro del sistema. Sirve como superclase para `Docente` y `Estudiante`, facilitando la reutilización de código y la implementación uniforme del manejo de resultados científicos.

Atributos

- `String nombre`: Nombre del investigador.
- `String apellidos`: Apellidos del investigador.
- `ArrayList<ResultadoInvestigativo> resultados`: Lista de resultados científicos asociados al investigador (artículos, ponencias, capítulos, etc.).

Métodos y su propósito

Constructor

- `Investigador(String nombre, String apellidos)`: Inicializa el objeto con nombre, apellidos y una lista vacía de resultados. Valida que el nombre no esté vacío ni contenga caracteres inválidos.

Gestión de resultados científicos

- `agregarResultado(ResultadoInvestigativo r)`: Agrega un resultado si no está duplicado.

- `removeResultado(ResultadoInvestigativo r)`: Elimina un resultado existente. Lanza excepciones si la lista está vacía o si el resultado no existe.
- `contieneResultado(ResultadoInvestigativo r)`: Verifica si un resultado ya está registrado.

Cálculo de desempeño

- `calcularPuntajeInvestigativo()`: Recorre los resultados científicos del investigador y suma el puntaje aportado por cada uno mediante el método `aportarPuntaje()`.

Creación de resultados científicos

- `crearArticulo(...)`: Crea un objeto `Articulo` con los datos dados y lo registra.
- `crearPonenciaEvento(...)`: Crea una `PonenciaEvento` y la agrega a los resultados.
- `crearCapituloLibro(...)`: Genera un `CapituloLibro` con su respectiva información y lo registra.

Estadísticas

- `getTotalArticulos()`: Retorna la cantidad total de artículos del investigador.
- `getTotalPonencias()`: Retorna la cantidad total de ponencias del investigador.
- `getTotalCapitulosLibros()`: Retorna la cantidad total de ponencias del investigador.

- `LíneaInvestigación`

La clase `LineaInvestigacion` representa un área temática dentro de un departamento donde se agrupan investigadores con intereses comunes. Permite gestionar la asociación de docentes y estudiantes a proyectos investigativos y localizar los autores de resultados científicos. Cada línea cuenta con un responsable que debe ser un docente.

Atributos

- String nombre: Nombre de la línea de investigación.
- ArrayList<Investigador> investigadores: Lista de investigadores (docentes o estudiantes) que forman parte de la línea.
- Docente responsable: Docente asignado como responsable de la línea.

Métodos y su propósito

Constructor

- LineaInvestigacion(String nombre, Docente responsable): Inicializa una línea de investigación con nombre y docente responsable. También crea la lista vacía de investigadores. Valida que el nombre no sea nulo, vacío ni contenga caracteres inválidos.

Gestión de investigadores

- agregarInvestigador(Investigador i): Agrega un nuevo investigador si no está duplicado y no es null.
- removerInvestigador(Investigador i): Elimina un investigador si está registrado. Lanza excepción si no existe o si el valor es null.
- contieneInvestigador(Investigador inv): Verifica si un investigador está contenido en la línea.

Localización de autor

- buscarInvestigadorDelResultado(ResultadoInvestigativo r): Recorre todos los investigadores asociados a la línea para determinar cuál de ellos tiene registrado el resultado científico dado. Devuelve el autor si se encuentra, o null en caso contrario.

Estadísticas

- getTotalArticulosPublicadosPorInvestigadores(): Retorna la cantidad total de artículos entre todos sus investigadores.
- getTotalPonenciasPublicadasPorInvestigadores(): Retorna la cantidad total de ponencias entre todos sus investigadores.
- getTotalCapitulosLibrosPublicadosPorInvestigadores(): Retorna la cantidad total de ponencias entre todos sus investigadores.
- getTotalResultadosPublicadosPorInvestigadores(): Retorna la cantidad total de resultados de la línea de investigación sumando los anteriores totales.

Método de comparación

- equals(Object o): Compara dos líneas de investigación en base a su nombre, permitiendo igualdad lógica.

- ResultadoInvestigativo

La clase ResultadoInvestigativo es una clase abstracta que define la estructura general para representar cualquier tipo de resultado derivado de la actividad científica o académica de un investigador. Su propósito es estandarizar el manejo de distintos productos investigativos, tales como artículos, ponencias y capítulos de libros.

Métodos abstractos

- String getNombrePublicacion(): Método que debe devolver el nombre o título del resultado específico.
- String getTipo(): Método que retorna el tipo concreto del resultado (por ejemplo: "Artículo", "Ponencia", "Capítulo").
- int aportarPuntaje(): Método que indica el valor numérico o puntuación asociada al resultado, útil para cálculos de rendimiento investigativo.

- Artículo

La clase Artículo representa un resultado investigativo correspondiente a una publicación científica o académica en una revista. Hereda de la clase abstracta ResultadoInvestigativo e implementa sus métodos para identificar el tipo de resultado, su nombre y el puntaje que aporta.

Atributos

- String titulo: Título del artículo.
- int numero: Número del artículo dentro de la publicación.
- int volumen: Volumen de la revista o serie donde se publicó.
- String anioPublicacion: Año en el que fue publicado el artículo (validado como cadena numérica).
- int paginas: Número de páginas del artículo.
- GrupoImpacto grupo: Clasificación del artículo según el impacto de la revista, el cual determina su puntaje.

Constructor

- Artículo(String titulo, int numero, int volumen, String anioPublicacion, int paginas, GrupoImpacto grupo): Inicializa el objeto validando cada parámetro para garantizar consistencia y significado.

Métodos sobrescritos

- `getNombrePublicacion()`: Devuelve el título del artículo.
- `getTipo()`: Retorna el tipo "Artículo".
- `aportarPuntaje()`: Devuelve el puntaje asociado según el grupo de impacto al que pertenece.

Comparación lógica

- `equals(Object o)`: Define igualdad lógica entre artículos, basada en título, número, volumen, año, páginas y grupo de impacto.

- `CapituloLibro`

La clase `CapituloLibro` representa un resultado investigativo concreto, correspondiente a la publicación de un capítulo en un libro académico. Forma parte del modelo como una subclase de `ResultadoInvestigativo`, lo que permite manejar este tipo de resultado de forma homogénea junto a otros como artículos y ponencias.

Atributos

- `String titulo`: Título del capítulo.
- `ArrayList<String> autores`: Lista de autores del capítulo.
- `ArrayList<String> nombresEditores`: Lista de editores responsables del libro.
- `String editorial`: Nombre de la editorial que publica el libro.
- `String ISSN`: Código ISSN del libro, validado como número de 8 dígitos.
- `String volumen`: Volumen del libro donde aparece el capítulo.
- `static int puntos`: Valor de puntaje fijo (3 puntos) que este tipo de resultado aporta.

Constructor

- `CapituloLibro(...)`: Inicializa todos los campos y realiza validaciones exhaustivas sobre el formato y contenido de los datos, previniendo errores por entradas vacías o inválidas.

Métodos sobrescritos

- `getNombrePublicacion()`: Retorna el título del capítulo.
- `getTipo()`: Devuelve la cadena "Capítulo de Libro" como descripción del tipo.
- `aportarPuntaje()`: Retorna el valor entero 3, como puntaje estándar de este tipo de publicación.

Método equals

- Se considera que dos capítulos son iguales si coinciden en título, editorial, ISSN y volumen. Esto garantiza una identificación precisa dentro de una colección.

- PonenciaEvento

La clase PonenciaEvento representa un resultado investigativo de tipo ponencia presentada en un evento académico o científico. Es una subclase de ResultadoInvestigativo, lo que permite manejar este resultado junto con otros tipos similares.

Atributos

- String nombre: nombre o título de la ponencia.
- LocalDate fecha: fecha en la que se presentó la ponencia.
- String anio: año extraído de la fecha, para facilitar consultas y comparaciones.
- String lugar: lugar donde se realizó el evento.
- String ISBN: código ISBN del evento o publicación relacionada, validado con 13 dígitos numéricos.
- static int puntos: puntaje fijo (1 punto) asignado a este tipo de resultado.

Constructor

- PonenciaEvento(String nombre, LocalDate fecha, String lugar, String ISBN): Inicializa todos los campos con validaciones estrictas para asegurar la integridad de los datos.

Métodos sobrescritos

- getTipo(): devuelve la cadena "Ponencia de Evento".
- getNombrePublicacion(): retorna el nombre de la ponencia.
- aportarPuntaje(): devuelve el puntaje asignado (1 punto).
- equals(Object o): considera dos ponencias iguales si coinciden en nombre, año, fecha, lugar y ISBN.

- Maestria

La clase Maestria modela un programa de posgrado que incluye un nombre, duración en meses, campo de estudio, cursos asociados y docentes matriculados. Facilita la gestión integral de la maestría, sus cursos y los docentes que participan.

Atributos

- String nombre: nombre de la maestría, solo letras y espacios.
- int duracionMeses: duración total del programa en meses, valor positivo.
- String campoEstudio: área o especialidad de la maestría, sin números ni caracteres especiales.
- ArrayList<CursoPosgrado> cursos: lista de cursos que conforman la maestría.
- ArrayList<Docente> matriculados: docentes matriculados en la maestría (deben no tener categoría científica).

Constructor

- Maestria(String nombre, int duracionMeses, String campoEstudio): Inicializa todos los atributos con validaciones para garantizar datos válidos.

Validaciones

- nombre y campoEstudio no pueden estar vacíos ni contener caracteres inválidos.
- duracionMeses debe ser un entero positivo.
- Para matricular un docente, debe tener categoría científica NINGUNA.

Métodos principales

- agregarMatriculado(Docente d): añade un docente a la lista de matriculados, lanzando excepciones si ya está o no cumple requisitos. Añade el docente a todos los cursos y elimina el registro de sus antiguos.
- eliminarRegistrosAntiguosCursos(Docente d, CursoPosgrado cursoPosgrado): Elimina el registro de cursos anteriores de los cursos de la matrícula en el docente.
- removerMatriculado(Docente d): elimina un docente matriculado y lo elimina de los cursos correspondientes.
- agregarCursoPosgrado(CursoPosgrado c) y removerCurso(CursoPosgrado c): gestión de cursos con validaciones de duplicación y existencia.
- crearCursoPosgrado(String tema, ArrayList<String> objetivos, int cantCreditos, Docente profesor): crea y agrega un curso nuevo a la maestría.
- getCreditosTotalesCursos(): suma los créditos de todos los cursos de la maestría.
- contieneCurso(CursoPosgrado c) y contieneMatriculado(Docente m): verifican existencia en las listas respectivas.

Sobrescrituras

- equals(Object o): compara maestrías por nombre, duración y campo de estudio.
- toString(): devuelve el nombre de la maestría.

- CursoPosgrado

La clase CursoPosgrado representa un curso específico de posgrado dentro de un programa académico. Gestiona la información fundamental del curso, sus objetivos, créditos asignados, el profesor que lo imparte y los docentes participantes.

Atributos

- String tema: tema principal del curso, validado para contener solo letras y espacios.
- ArrayList<String> objetivos: lista de objetivos del curso, no puede estar vacía ni contener cadenas vacías o inválidas.
- int cantCreditos: cantidad de créditos que otorga el curso, debe ser un entero positivo.
- ArrayList<Docente> participantes: lista de docentes que participan en el curso, excluyendo al profesor.
- Docente profesor: docente que imparte el curso, debe tener categoría científica de Doctor.

Constructor

- CursoPosgrado(String tema, ArrayList<String> objetivos, int cantCreditos, Docente profesor): Inicializa los atributos con validaciones estrictas, creando una instancia válida del curso.

Métodos principales

- agregarParticipante(Docente d): Añade un docente a la lista de participantes, excepto el profesor. Lanza excepciones si el docente es nulo, es el profesor, o ya está registrado.
- removerParticipante(Docente d): Remueve un docente participante, verificando que la lista no esté vacía y que el docente exista en ella.
- emitirNota(Docente evaluador, Docente participante, int nota): El profesor que imparte el curso puede evaluar a los participantes asignándoles una nota entre 2 y 5. Las notas ≥ 3 otorgan los créditos del curso, inferiores a 3 no. La nota se registra creando un objeto CursoRecibido para el participante y si existía un anterior registro del mismo curso, se reemplaza con el nuevo.

Sobrescritura

- equals(Object c): Dos cursos se consideran iguales si sus objetivos, cantidad de créditos, tema y profesor coinciden.

- CursoRecibido

La clase CursoRecibido representa la información sobre un curso de posgrado que un docente ha completado, incluyendo la nota obtenida, los créditos recibidos y el tema del curso.

Atributos

- int nota: la calificación obtenida en el curso, representada como un entero.
- int creditsRecibidos: la cantidad de créditos académicos obtenidos tras aprobar el curso.
- String temaCurso: el tema o título del curso recibido.

Constructor

- CursoRecibido(int nota, int creditsRecibidos, String temaCurso): Inicializa la instancia con la nota, créditos y tema del curso. No incluye validaciones explícitas, por lo que se asume que estas se realizan antes de crear el objeto.

Getters

- getNota(): devuelve la nota obtenida.
- getCreditsRecibidos(): devuelve los créditos recibidos por el curso.
- getTemaCurso(): devuelve el tema o nombre del curso.

Métodos

- equals(Object c): redefine la igualdad entre objetos, considerando que dos instancias de CursoRecibido son iguales si tienen igual tema del curso.

7. Explicación de la división por capas y paquetes

En el proyecto hay presentes varios paquetes:

- Excepciones: Aquí se crearon excepciones personalizadas heredando todas de RuntimeException para generalizarlas todas en un try catch para validar.
- Logica: Aquí se encuentran presentes todas las clases de la solución de la problemática.
- Interfaz: Todas las clases relacionadas con la interfaz gráfica.
- Login: Aquí se encuentran presentes las clases relacionadas con el sistema de registro de la aplicación.
- Iconos: Paquete para las imágenes del proyecto.
- Principal: Aquí se encuentra la clase Inicializadora que le da inicio al programa.